AIRSCOPE ANALYTICS 2.0 • ARCHITECTURE & UI/UX BLUEPRINT

AirScope Analytics

Enterprise Environmental Intelligence Platform with 3D Spatial Pollution Corridors & Machine-Learning Analytics



Core Mission: Transform fragmented multi-source environmental sensor data into an intuitive, high-fidelity spatial dashboard combining Three.js 3D isometric city models, localized meteorological badges, rigorous SARIMAX forecasting, and traceable database persistence.

ARCHITECTURE BASELINE
React 19 • FastAPI • SQLAlchemy • Three.js

TARGET UX AESTHETIC
Modern Clean Light / Glassmorphic HUD

DOCUMENT STATUS
Approved Specification & Phased Execution Plan

1. Executive Vision & UI/UX Breakdown

The target design introduces an intuitive spatial paradigm for air quality monitoring. Instead of flat 2D numbers, users navigate a 3D isometric urban model where pollution concentrations are rendered as **volumetric building corridor heatmaps** alongside floating real-time meteorological badges.

🌆 1. Immersive 3D Pollution Mesh

Three.js WebGL canvas displaying isometric city blocks. Buildings within high-pollution plumes are shaded along an AQI gradient: Green (Good $le 12,mu ext{g/m}^3$), Yellow (Moderate $12-35,mu ext{g/m}^3$), and Orange/Red (Unhealthy $ge 35,mu ext{g/m}^3$). Camera supports smooth panning, 2D/3D toggle, and tilt.

📍 2. Floating Meteorological Nodes

Spatial world-to-screen projected badges hover directly over geographic sensor coordinates. They deliver immediate localized environmental readings: **Rainfall** (e.g. 0 mm), **Humidity** (84%), **Temperature** (23°C), and **Visibility** (5 km).

📊 3. Glassmorphic Analytics Drawer

A floating bottom container housing three core widgets:
• **AQI Timeline**: Hourly/Daily/Monthly segmented bar and line chart.
• **Pollution Sources**: Apportionment breakdown (Traffic 52%, Construction 20%, Industry 14%).
• **Wind Compass**: Directional rose & dispersion velocity meter (7.5 km/h, 1 m/s).

🧭 4. Unified Sidebar & Location Bar

A fixed vertical left dock providing rapid navigation between Home, Dashboard, Trends, City Comparison, 3D Map, AI Forecasting, and Reports. A top header hosts the quick-search location dropdown and ambient weather widget.

2. System Layer Architecture

ARCHITECTURE LAYER	COMPONENTS & FRAMEWORKS	FUNCTIONAL RESPONSIBILITIES
1. Presentation Layer	React 19, Vite 7, Tailwind CSS, Three.js, Recharts, Framer Motion	Interactive 3D city canvas, glassmorphic HUD cards, responsive analytics charts, printable compliance reports, JWT session context.
2. Application Layer	FastAPI, Uvicorn, Pydantic v2, Python-JOSE / PyJWT, Passlib	RESTful API endpoints, RBAC subscription tier gates (Free, Pro, Enterprise), request logging middleware, CORS security.
3. Data & Storage Layer	MySQL 8.0, SQLAlchemy 2.0 ORM, PyMySQL (with SQLite fallback)	Persists canonical geocodes, historical hourly sensor readings with source attribution, user credentials, and session refresh tokens.
4. AI & Analytics Layer	Statsmodels (SARIMAX), Scikit-Learn, Ollama (Gemma3:4b), ReportLab	Hourly time-series forecasting with 80% confidence bands, rolling backtesting, natural-language agent tool planning, server PDF rendering.

3. Application Routing & Access Boundaries

The application implements strict Role-Based Access Control (RBAC) enforced both at the client router (`RequireAuth`) and backend middleware (`tiers.py`):

ROUTE PATH	VIEW COMPONENT	TIER REQUIRED	PRIMARY FEATURES
`/home`	`Home.jsx`	PUBLIC	Marketing landing page, LiquidEther 3D WebGL shader hero, feature showcase, pricing tiers.
`/signin` / `/signup`	`Signin.jsx` , `Signup.jsx`	PUBLIC	Authentication entrypoints, bcrypt password hashing, HTTP-only JWT cookie management.
`/dashboard`	`DashboardView.jsx`	FREE	**Target UI/UX**: 3D city pollution map, floating met badges, live AQI drawer, single-city data scraping.
`/trends`	`TrendsView.jsx`	PRO	Historical diurnal pollution patterns, 24h moving averages, multi-week data tables.
`/compare`	`ComparisonView.jsx`	PRO	Multi-city PM2.5/PM10 comparison, best vs. worst ranking, part-to-whole pollutant delta.
`/forecast`	`ForecastView.jsx`	PRO	SARIMAX hourly predictions, 80% confidence interval bands, MAE/RMSE rolling backtest validation.
`/ai-insights`	`AgentView.jsx`	ENTERPRISE	LLM planning agent (Gemma3:4b) with reflection critic pattern, multi-step execution traces.
`/reports`	`ReportsView.jsx`	PRO	Server-side ReportLab PDF report generation with chart rasterization & interactive print preview.
`/settings`	`SettingsView.jsx`	FREE	Account profile, active tier status, API token management, theme preferences.

4. End-to-End Data Ingestion & Traceability Pipeline

> **Data Traceability Rule:** All raw measurements must preserve source attribution (`open-meteo` , `openaq` , `iqair` , `waqi`) and never overwrite existing historical hourly slots. Cleaned consensus signals are stored under `source = 'aggregated'` .

1. **Geocoding Resolution**: City names are resolved via Open-Meteo Geocoding API and permanently cached in the `geocodes` table.
2. **Multi-Source Scrape Dispatch**: Primary hourly readings fetched from Open-Meteo REST. Supplementary stations scraped from OpenAQ, WAQI, and IQAir.
3. **Normalization**: All timestamps converted to ISO-8601 UTC hourly boundaries (`YYYY-MM-DD HH:00:00`).
4. **Outlier Trimming**: Z-score ($pm 3.0sigma$) or IQR ($1.5 imes ext{IQR}$) filters trim anomalous sensor spikes.
5. **Weighted Aggregation**: Sources merged by timestamp using configured weights (`AGG_WEIGHTS`) and stored as `source = 'aggregated'` .
6. **Time-Series Extraction**: Analytics endpoints query MySQL directly, returning authentic historical series to the frontend without client-side mock synthesis.

5. Database Schema Declarations (SQLAlchemy ORM)

The domain schema is explicitly defined in `backend/app/models.py`, resolving the critical blocker where `geocodes` and `measurements` were previously missing:

```
# backend/app/models.py
from datetime import datetime
from sqlalchemy import (
    Column, BigInteger, String, Float, DateTime,
    Enum, ForeignKey, UniqueConstraint, Index
)
from sqlalchemy.orm import declarative_base, relationship

Base = declarative_base()

class User(Base):
    __tablename__ = "users"
    id = Column(BigInteger, primary_key=True, index=True, autoincrement=True)
    email = Column(String(190), unique=True, index=True, nullable=False)
    password_hash = Column(String(255), nullable=False)
    plan = Column(Enum('free', 'pro', 'enterprise', name='plan_enum'), default='free', nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow, nullable=False)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow, nullable=False)
    last_login = Column(DateTime, nullable=True)
    refresh_tokens = relationship("RefreshToken", back_populates="user", cascade="all, delete-orphan")

class RefreshToken(Base):
    __tablename__ = "refresh_tokens"
    id = Column(BigInteger, primary_key=True, index=True, autoincrement=True)
    user_id = Column(BigInteger, ForeignKey("users.id", ondelete="CASCADE"), nullable=False)
    token_hash = Column(String(255), nullable=False)
    expires_at = Column(DateTime, nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow, nullable=False)
    user = relationship("User", back_populates="refresh_tokens")

class Geocode(Base):
    __tablename__ = "geocodes"
    city = Column(String(100), primary_key=True)
    latitude = Column(Float, nullable=False)
    longitude = Column(Float, nullable=False)
    country = Column(String(100), nullable=True)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

class Measurement(Base):
    __tablename__ = "measurements"
    id = Column(BigInteger, primary_key=True, autoincrement=True)
    ts = Column(DateTime, nullable=False, index=True)
    city = Column(String(100), nullable=False, index=True)
    latitude = Column(Float, nullable=True)
    longitude = Column(Float, nullable=True)
    pm25 = Column(Float, nullable=True)
    pm10 = Column(Float, nullable=True)
    source = Column(String(50), nullable=False, index=True)

    __table_args__ = (
        UniqueConstraint("ts", "city", "source", name="uq_ts_city_source"),
        Index("idx_city_source_ts", "city", "source", "ts"),
    )
```

6. REST API Interface Contracts

📡 POST /scrape

```
// Request
{
  "city": "London",
  "days": 7,
  "sources": ["open-meteo"]
}

// Response
{
  "ok": true,
  "city": "London",
  "inserted": 168,
  "series": [
    { "ts": "2026-10-14 00:00:00", "pm25":
14.2, "pm10": 22.1 }
  ]
}
```

📊 POST /compare

```
// Request
{
  "cities": ["London", "Paris"],
  "days": 7
}

// Response
{
  "ok": true,
  "best": "Paris",
  "worst": "London",
  "byCity": {
    "London": {
      "mean_pm25": 24.5, "min_pm25": 8.1,
"max_pm25": 48.2,
      "series": [{ "ts": "2026-10-14
00:00:00", "pm25": 24.5 }]
    }
  }
}
```

7. Phased Implementation Roadmap

Every major feature has a defined dependency order to preserve existing working systems while executing the visual and structural transformation:

Phase 1: Foundation & Data Stability (Blockers)

- ☑ Complete Architectural Audit & Deficiency Analysis (Finished)
- ☑ AirScope Analytics System Specification & UI/UX Blueprint (Finished)
- ☐ **Database Layer**: Add `Measurement` and `Geocode` models to `models.py`; add SQLite automatic development fallback in `db.py`; move `create_all` to FastAPI lifespan.
- ☐ **Frontend Layout**: Move `DataCollectionTab.jsx` to `src/components/data-collection/`; fix relative imports.
- ☐ **Auth Error Handling**: Fix `auth.py` login handler to re-raise `HTTPException` so 401 Unauthorized is returned properly instead of 500.
- ☐ **Dependency Sanitation**: Save `requirements.txt` as clean UTF-8; replace `jsonwebtoken` in `package.json` with `jwt-decode`.

Phase 2: Data Pipeline Traceability & Backend Integration

- ☐ **Backend Series Payloads**: Update `compare_logic` in `compare.py` and `scraper.py` to return authentic hourly series.
- ☐ **Purge Synthetic Mock Data**: Remove all `Math.random()`, `Math.sin()`, and `Math.cos()` generators from `Workspace.jsx`.
- ☐ **Unified API Client**: Consolidate dual `fetch` / `axios` clients into a single Axios client with dynamic user-plan injection from `AuthContext`.

Phase 3: AirScope 3D Environmental Dashboard (Target UI/UX)

- ☐ **Three.js City Canvas**: Build `PollutionMap3D.jsx` with isometric building blocks and AQI gradient shaders (Green/Yellow/Red).
- ☐ **Floating Spatial Badges**: Build `FloatingNodePin.jsx` displaying projected Rainfall, Humidity, Temperature, and Visibility.
- ☐ **Glassmorphic Analytics Drawer**: Build `AnalyticsDrawer.jsx` containing AQI threshold bar timeline, pollution source breakdown, and wind compass.
- ☐ **AppShell & Sidebar**: Implement vertical icon dock and top location selector with real-time weather badge.

Phase 4: Reporting, AI Agent & Verification

- ☐ **ReportLab PDF Pipeline**: Wire "Download PDF Report" buttons directly to backend `/report/generate` via `chartCapture.js`.
- ☐ **AI Agent Critic Pattern**: Connect `plan_with_critic` in `agent.py` to enforce prompt security and capability filtering.
- ☐ **Prophet Pruning**: Cleanly prune orphaned `forecast_prophet.py` to standardize on SARIMAX.
- ☐ **Automated Test Harness**: Implement Pytest route tests (Auth, Scrape, Compare, Forecast) and Vitest frontend sanity tests.